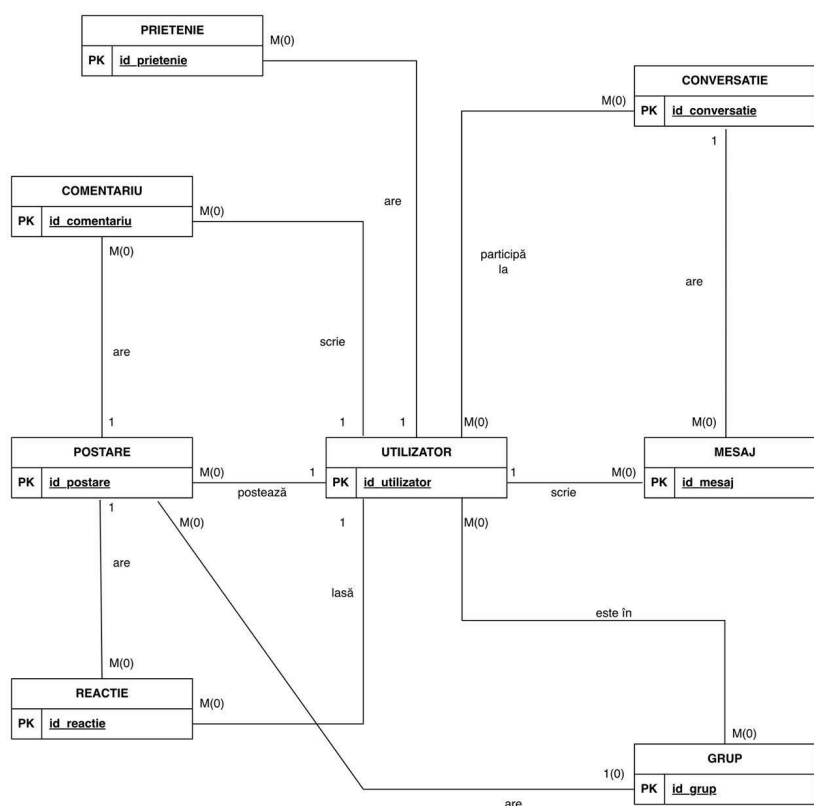


Raport de activitate: Dezvoltarea aplicațiilor web

1. Planing

Pentru a comunica într-un mod deschis și facil, prima planificare a fost pe discord, într-un apel în care am discutat de ideile pe care urma să le implementăm și de ce lucruri am vrea propriu-zis să facem. Ulterior pasului inițial, am continuat prin a vorbi la facultate, pe instagram și chiar și la telefon, în special după cursurile sau laboratoarele de DAW în care primeam idei noi pe care am fi vrut să le implementăm.

Am început prin a crea o diagramă ERD care să ne ajute în gândirea părții de backend a proiectului. Cu ajutorul acesteia, deși nu este în formă finală și nu reprezintă în totalitate proiectul implementat, am reușit să creăm modelele și să ne gândim la entitățile ce urma să le creăm mult mai ușor.



Ca și mod de lucru am ales să creăm o organizație pe github și, desigur, un [repository](#) pentru proiect. Aici am reușit să lucrăm în paralel, fiecare dintre noi implementând ceva diferit în același timp. De multe ori ne aflam pe discord sau pe un apel și lucram la proiect întrebându-ne ce moduri de implementare am putea folosi, ce resursă didactică să consultăm pentru următoarea parte a proiectului. Per total, în ciuda unor merge conflicturi, am reușit să lucrăm într-un mod eficient și să fim ambii mult mai productivi.

2. Comunicare și împărțirea taskurilor

Fiind deja prieteni buni și colegi de grupă ne-a fost extrem de ușor să ținem legătura și să comunicăm despre task-urile rezolvate sau în lucru. Nu am avut conflicte sau impedimente, ambii lucrând într-un ritm similar și organizându-ne pe tot parcursul colaborării.

Pentru împărțirea inițială a task-urilor, după ce am întrebat un LLM care ar fi cea mai bună metodă de a le organiza astfel încât fiecare să contribuie într-un mod egal atât la partea de backend, cât și la cea de frontend, am căzut de acord asupra asignării conform imaginilor de mai jos.

De fiecare dată când apărea vreo problemă în ceea ce înseamnă implicarea pe moment în proiect fiecare din noi îl atenționa pe celălalt fie atunci când ne întâlneam la facultate, fie printr-un apel sau mesaj.

3. Productivitate

Propunerea noastră inițială a fost să atribuim deadline-uri fiecărui sprint și să le respectăm cu strictețe... însă nu am reușit să ne ținem constant de acestea... De asemenea inițial aveam o cu totul altă idee pentru acest proiect (dovadă că se numește OSSocial) pe care nu am reușit să o implementăm în timpul alocat.

Un lucru important de menționat totuși este că productivitatea amândurora creștea exponențial în momentul în care lucram “cot la cot” (pe discord :)), iar atunci când lucram chiar o făceam într-un mod foarte productiv (în paralel și implementând ambii feature-uri atât de frontend cât și de backend).

4. Impedimente

Printre impedimentele tehnice pe care le-am avut apar o suită de bug-uri, presărate cu diferite merge conflict-uri și erori apărute pe branch-ul main din cauza integrării lor greșite. :(

Printre bug-uri se numără erorile ce apăreau în momentul folosirii diferitor naming convention-uri de către fiecare. De foarte multe ori dădeam merge la un branch doar ca să realizăm că celălalt a folosit un alt nume pentru userManager (_userManager) sau pentru baza de date și normal că toată aplicația pica. Un bug constant și deranjant (care a fost motivul pentru care niciunul dintre noi nu a lucrat pe windows, ci pe MacOS sau Linux Mint respectiv) este cel ce a apărut când pachetul dotnet entity framework arăta “erori fantomă” în IDE. (Plus cumva nici scaffolding items nu funcționau așa că ambii am ales să lucrăm pe Rider).

Totuși, cel mai greu de contracarat impediment pe care l-am întâlnit pe parcursul dezvoltării aplicației a fost începerea sa propriu-zisă. Deși lucrasem înainte de vacanță, momentumul pe care îl obținusem atunci nu a reușit să supraviețuiască după aceasta, în special pentru că în prima săptămână am avut 3 deadline-uri, un test și un examen.

5. Metodologia de lucru

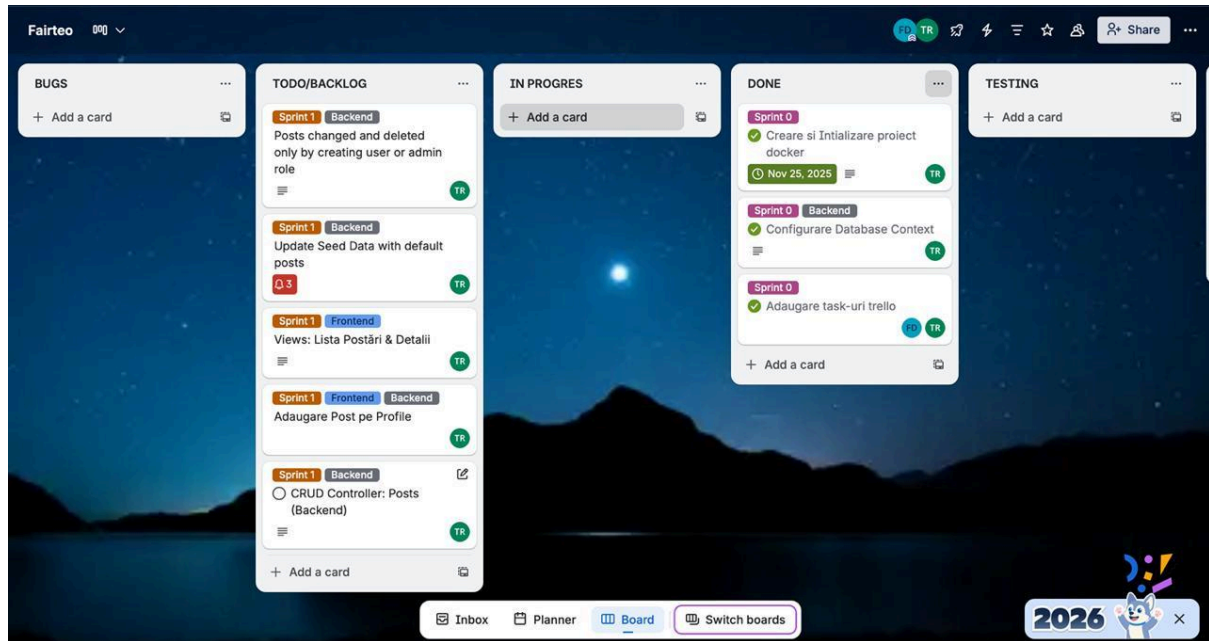
Un mod foarte intuitiv de a lucra a fost folosind metodologia Agile. Astfel organizându-ne task-urile într-un mod intuitiv și ușor de înțeles. Având cinci sprinturi (dintre care primul a fost împărțirea task-urilor pe Trello și crearea proiectului cu Docker).

Am folosit docker pentru a rula facil aplicația pe orice sistem de operare (acest proiect chiar a fost rulat pe absolut tot la un moment dat fiindcă inițial a fost creat pe Windows, după aceea pe Linux, iar unul din noi a folosit exclusiv Mac). Pentru controlul versiunilor am folosit GitHub (menționat și mai sus), iar pentru împărțirea și vizualizarea task-urilor am folosit site-ul recomandat la curs, anume Trello.

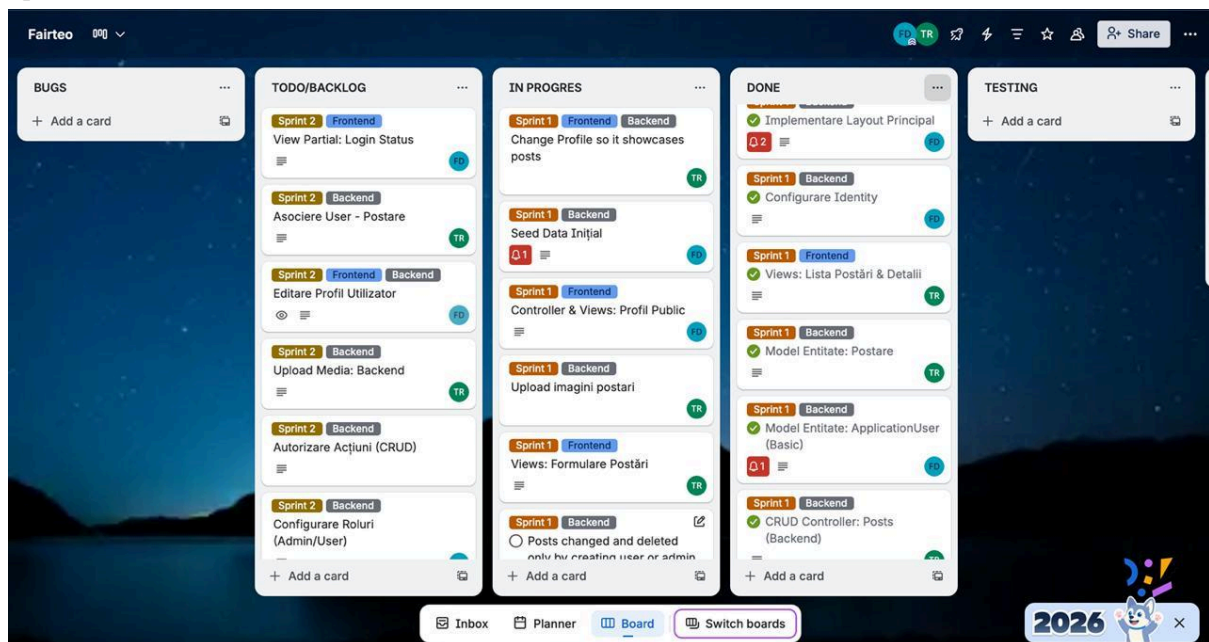
6. Progres sprint

În Sprintul 0 am creat board-ul de trello și fiecare task de pe acesta. Am avut o sesiune de brainstorming în care am găsit idei bune pentru a crea o aplicație web originală ca interfață (pe care din

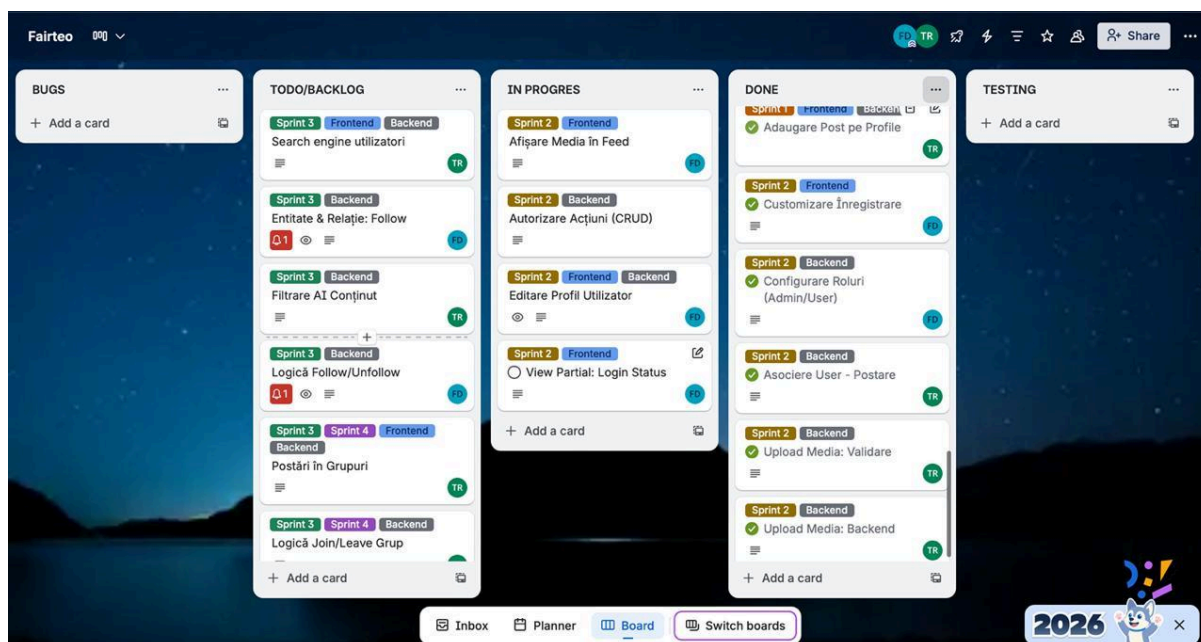
nefericire nu am reușit să le ducem la capăt ne pare rău) și ne-am decis în ce mod să împărțim task-urile.



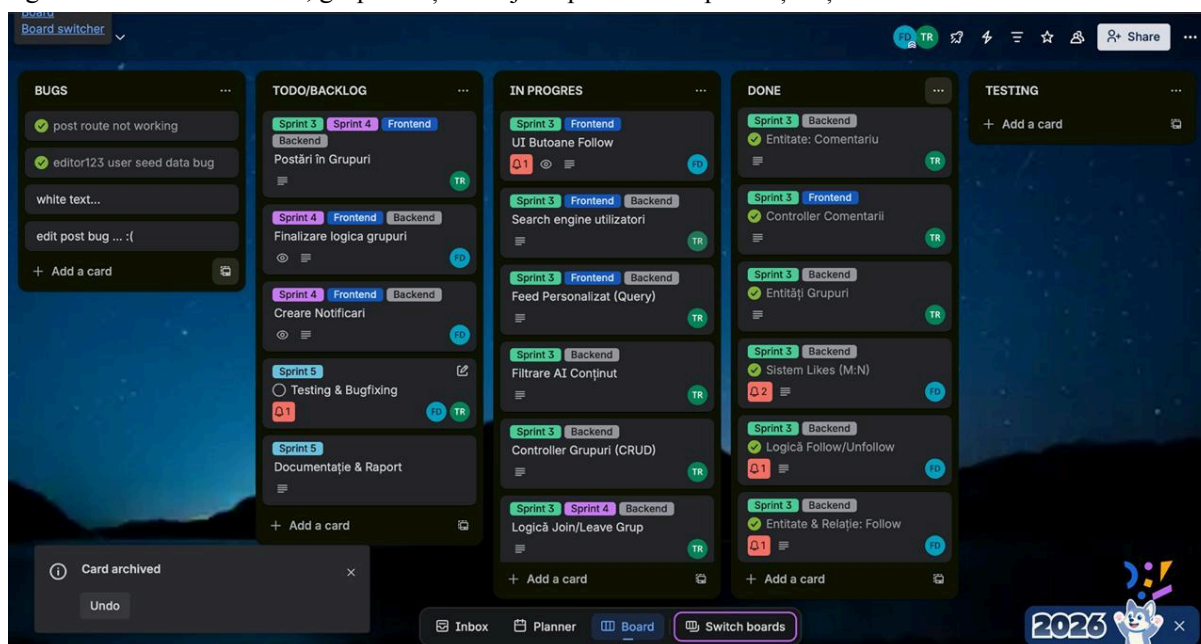
Sprintul 1 a reprezentat pentru noi începutul propriu-zis, în acesta am creat postările, am actualizat modelul IdentityUser cu datele necesare nouă și am adăugat un Seed Data pentru a ușura partea de testing a proiectului.



Sprintul 2 a reprezentat lucruri precum: implementarea postărilor pe fiecare profil, uploadat-ul de imagini sau videoclipuri, autorizarea de acțiuni și configurarea rolurilor.



Sprintul 3 conține lucrurile puțin mai complexe precum filtrarea conținutului folosind AI-ul, logica de Follow/Unfollow, grupurile și mesajele/ postările de pe ele și ieșirea din acestea.



Sprintul 4 și 5 reprezintă finalizarea proiectului în care creat documentația, am făcut bug fixing, am schimbat părți din frontend și am creat notificările.

7. Lucru în echipă

În semestrele anterioare, de fiecare dată când mai primeam câte un proiect în echipă alegeam să lucrăm împreună așa că ne-a fost extrem de ușor să colaborăm, dat fiindcă știam deja cum lucrează celălalt. Ambii am avut un aport de lucru similar și am contribuit în aceeași măsură.

Partea de testing a aplicației a fost făcută individual de către fiecare, însă asta nu însemna că, după un merge, celălalt nu reușea să mai găsească și să mai rezolve câte un bug apărut prin codul celuiilalt. (La un moment dat am avut chiar un moment în care ambii am găsit bug-uri în exact același moment...)

8. Integrarea Companionului AI

Pentru a asigura faptul că niciun utilizator nu poate crea postări sau comentarii cu conotații negative am integrat companionul AI în aplicație, folosind serviciul oferit de Google, anume Gemini (Flash Lite).

Implementarea companionului se realizează prin serviciul *ContentAnalysisService* care facilitează comunicarea dintre aplicația noastră cu API-ul extern prin cereri HTTP. Acesta interpretează ulterior răspunsul oferit de către AI și ajută la blocarea sau acceptarea unui text introdus de către utilizator.

Pașii pe care îi urmează codul sunt următorii:

- atunci când un utilizator vrea să posteze un comentariu sau o postare textul introdus este introdus către serviciul *AnalyzeContentAsync*
- partea de prompt engineering creează textul ce urmează să fie trimis propriu-zis către
- odată trimisă cererea către API-ul Google răspunsul este analizat. Dacă modelul AI determină că textul nu este bun atunci acesta este respins, iar publicarea este întreruptă
- în final utilizatorul primește un răspuns tot generat de AI!

```
var prompt = `You are a content analysis assistant. Analyze the content of the given text and respond ONLY with a JSON object in this exact format:
{"IsAccepted": true, "Reason": "explanation string"}

Rules:
- IsAccepted must be exactly one of: true, false
- If IsAccepted is false, provide a Reason for rejection in the explanation string
- the explanation string has to be explicit and shorter than 450 characters
- If IsAccepted is true, Reason can be an empty string
- IsAccepted must be a JSON boolean: true or false (NOT a string "true")
- SHORT CONTENT RULE: Short text, single words, or neutral statements (e.g., "Hi", "Update", "Cool") are ACCEPTABLE (IsAccepted: true) unless they are explicitly vulgar/offensive.
- EMPTY CONTENT RULE: text can be empty
- Do not include any other text, only the JSON object

Analyze the content of this comment: "{text}"`;
```

```
public class ContentResult
{
    // daca e acceptat sau nu mesajul
    public bool IsAccepted { get; set; }

    // daca nu e acceptat, motivul
    public string? Reason { get; set; }

    public bool Success { get; set; }

    public string? ErrorMessage { get; set; }
}
```